Name: Armaan Patnaik
Ly 28(2021)
Roll no. - 2223448

HPCL ASSIGNMENT 2

Q.1

Cloud Shell Editor

cloudshell

GNU nano 7.2

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

#define NUM_THREADS 4
#define INCREMENTS 100000

long counter = 0;
pthread_mutex_t mutex;

/* Function to get current time in milliseconds */
double get_time_ms() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1000.0 + ts.tv_nsec / 1000000.0;
}

/* Sequential execution */
void sequential_execution() {
    counter = 0;
    for (int i = 0; i < NUM_THREADS * INCREMENTS; i++) {
        counter++;
    }
}

/* Thread function without mutex */
void *increment_without_mutex(void *arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        counter++;
    }
    return NULL;
}

/* Thread function with mutex */
void *increment_with_mutex(void *arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        pthread_mutex_lock(&mutex);
        counter++;
```

Cloud Shell Editor

cloudshell

GNU nano 7.2

```c
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

/* Run threads without mutex */
double run_without_mutex(long *final_counter) {
    pthread_t threads[NUM_THREADS];
    counter = 0;

    double start = get_time_ms();

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL,
                       increment_without_mutex, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    double end = get_time_ms();

    *final_counter = counter;

    return end - start;
}

/* Run threads with mutex */
double run_with_mutex(long *final_counter) {
    pthread_t threads[NUM_THREADS];
    counter = 0;

    double start = get_time_ms();

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL,
                       increment_with_mutex, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
```

Cloud Shell Editor

cloudshell

GNU nano 7.2

```c
    double end = get_time_ms();

    *final_counter = counter;

    return end - start;
}

int main() {
    long result;
    double start, end;

    pthread_mutex_init(&mutex, NULL);

    printf("================================================\n");
    printf("QUESTION 1: RACE CONDITIONS AND MUTEX LOCKS\n");
    printf("================================================\n\n");

    printf("Expected result: %d\n\n",
           NUM_THREADS * INCREMENTS);

    /* Part A: Run without synchronization 5 times */

    printf("PART A: WITHOUT SYNCHRONIZATION\n");
    printf("------------------------------------------------\n");

    for (int i = 1; i <= 5; i++) {
        double time_taken = run_without_mutex(&result);

        printf("Run %d: Counter = %ld | Time = %.3f ms\n",
               i, result, time_taken);
    }

    /* Part B: Run with mutex 5 times */

    printf("\nPART B: USING MUTEX\n");
    printf("------------------------------------------------\n");

    for (int i = 1; i <= 5; i++) {
        double time_taken = run_with_mutex(&result);

        printf("Run %d: Counter = %ld | Time = %.3f ms\n",
               i, result, time_taken);
    }
```

Cloud Shell Editor

cloudshell

GNU nano 7.2

```c
    for (int i = 1; i <= 5; i++) {
        double time_taken = run_with_mutex(&result);

        printf("Run %d: Counter = %ld | Time = %.3f ms\n",
               i, result, time_taken);
    }

    /* Part C: Performance comparison */

    printf("\nPART C: PERFORMANCE COMPARISON\n");
    printf("------------------------------------------------\n");

    counter = 0;

    start = get_time_ms();
    sequential_execution();
    end = get_time_ms();

    printf("Sequential Execution:\n");
    printf("Counter = %ld | Time = %.3f ms\n\n",
           counter, end - start);

    double no_mutex_time = run_without_mutex(&result);

    printf("Multithreaded WITHOUT Mutex:\n");
    printf("Counter = %ld | Time = %.3f ms\n\n",
           result, no_mutex_time);

    double mutex_time = run_with_mutex(&result);

    printf("Multithreaded WITH Mutex:\n");
    printf("Counter = %ld | Time = %.3f ms\n",
           result, mutex_time);

    pthread_mutex_destroy(&mutex);

    printf("\n================================================\n");
    printf("PROGRAM COMPLETED\n");
    printf("================================================\n");

    return 0;
}
```

Name: Armaan Patnaik
Ly 28(2021)
Roll no. - 2223448

o/p –

```
armaanpatnaik2004@cloudshell:~$ gcc question1.c -o question1 -pthread -O0
armaanpatnaik2004@cloudshell:~$ ./question1
============================================
QUESTION 1: RACE CONDITIONS AND MUTEX LOCKS
============================================

Expected result: 400000

PART A: WITHOUT SYNCHRONIZATION
--------------------------------------------
Run 1: Counter = 337303 | Time = 1.965 ms
Run 2: Counter = 324300 | Time = 1.788 ms
Run 3: Counter = 323702 | Time = 1.841 ms
Run 4: Counter = 314960 | Time = 1.908 ms
Run 5: Counter = 311766 | Time = 1.864 ms

PART B: USING MUTEX
--------------------------------------------
Run 1: Counter = 400000 | Time = 34.470 ms
Run 2: Counter = 400000 | Time = 33.583 ms
Run 3: Counter = 400000 | Time = 35.014 ms
Run 4: Counter = 400000 | Time = 15.769 ms
Run 5: Counter = 400000 | Time = 32.978 ms

PART C: PERFORMANCE COMPARISON
--------------------------------------------
Sequential Execution:
Counter = 400000 | Time = 1.036 ms

Multithreaded WITHOUT Mutex:
Counter = 331701 | Time = 1.844 ms

Multithreaded WITH Mutex:
Counter = 400000 | Time = 32.893 ms

============================================
PROGRAM COMPLETED
============================================
armaanpatnaik2004@cloudshell:~$
```

Name: Armaan Patnaik
Ly 28(2021)
Roll no. - 2223448

Q.2

Cloud Shell Editor

cloudshell

GNU nano 7.2

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define BUFFER_SIZE 5
#define PRODUCERS 2
#define CONSUMERS 2
#define ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0, count = 0;

pthread_mutex_t mutex;
sem_t empty, full;

int producer_delay = 100000;
int consumer_delay = 100000;

void *producer(void *arg)
{
    int id = *(int *)arg;

    for (int i = 0; i < ITEMS; i++) {
        int item = rand() % 100;

        usleep(producer_delay);
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        count++;

        printf("Producer %d produced %d | Buffer count: %d\n",
               id, item, count);

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
```

Cloud Shell Editor

cloudshell

GNU nano 7.2

```c
    return NULL;
}

void *consumer(void *arg)
{
    int id = *(int *)arg;

    for (int i = 0; i < ITEMS; i++) {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;

        printf("Consumer %d consumed %d | Buffer count: %d\n",
               id, item, count);

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        usleep(consumer_delay);
    }

    return NULL;
}

int main(int argc, char *argv[])
{
    pthread_t p[PRODUCERS], c[CONSUMERS];
    int pid[] = {1, 2};
    int cid[] = {1, 2};

    srand(time(NULL));

    /* 1 = equal, 2 = producer faster, 3 = consumer faster */
    if (argc > 1) {
        int exp = atoi(argv[1]);

        if (exp == 2) {
            producer_delay = 20000;
            consumer_delay = 200000;
```

Cloud Shell Editor

cloudshell

GNU nano 7.2

```c
{
    pthread_t p[PRODUCERS], c[CONSUMERS];
    int pid[] = {1, 2};
    int cid[] = {1, 2};

    srand(time(NULL));

    /* 1 = equal, 2 = producer faster, 3 = consumer faster */
    if (argc > 1) {
        int exp = atoi(argv[1]);

        if (exp == 2) {
            producer_delay = 20000;
            consumer_delay = 200000;
        }
        else if (exp == 3) {
            producer_delay = 200000;
            consumer_delay = 20000;
        }
    }

    pthread_mutex_init(&mutex, NULL);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    for (int i = 0; i < PRODUCERS; i++)
        pthread_create(&p[i], NULL, producer, &pid[i]);

    for (int i = 0; i < CONSUMERS; i++)
        pthread_create(&c[i], NULL, consumer, &cid[i]);

    for (int i = 0; i < PRODUCERS; i++)
        pthread_join(p[i], NULL);

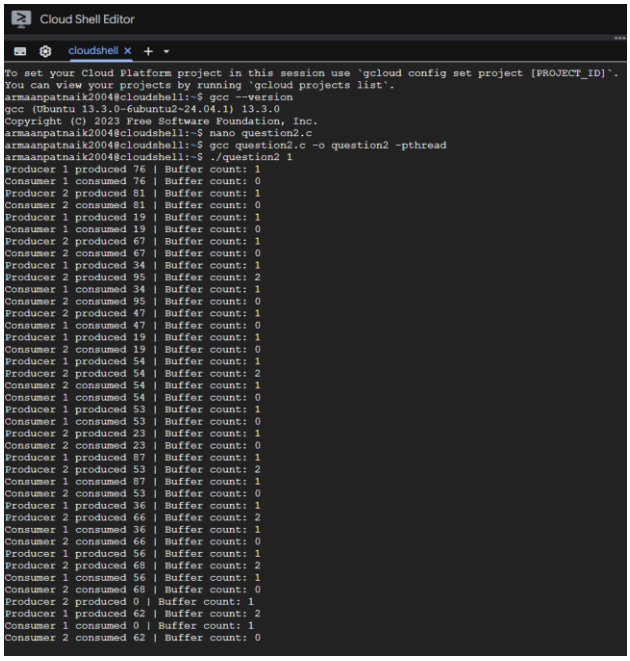
    for (int i = 0; i < CONSUMERS; i++)
        pthread_join(c[i], NULL);

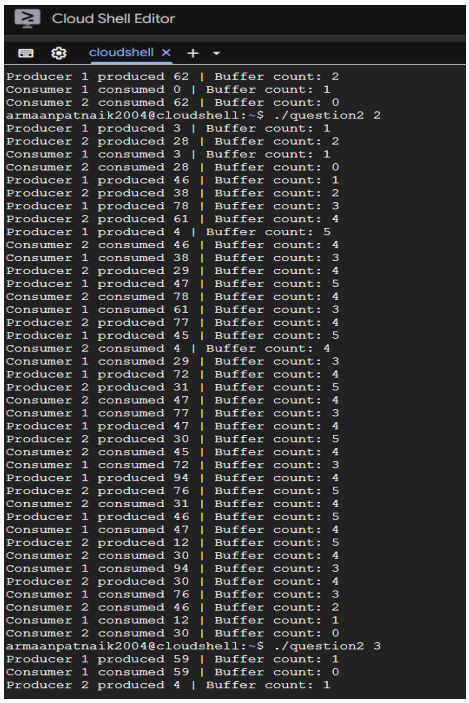
    pthread_mutex_destroy(&mutex);
    sem_destroy(&empty);
    sem_destroy(&full);

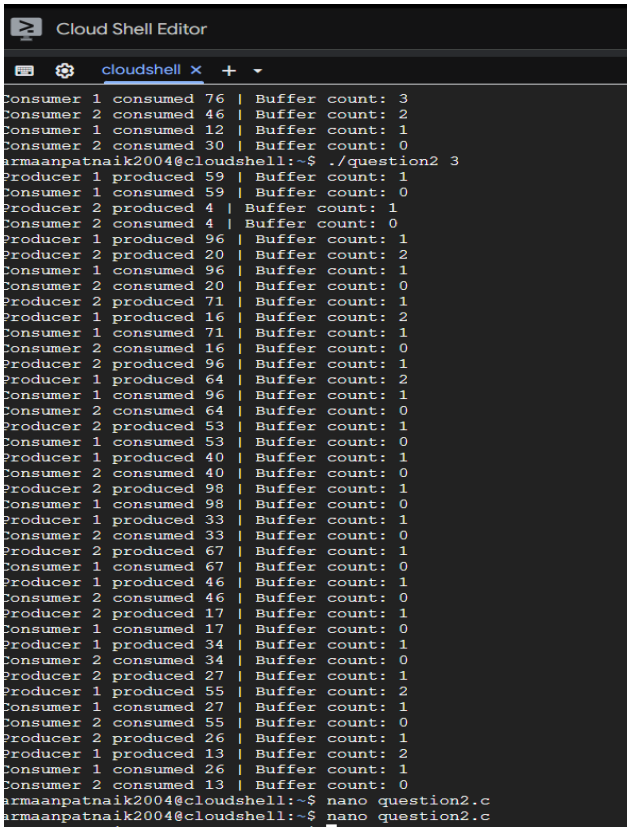
    return 0;
}
```

Name: Armaan Patnaik
Ly 28(2021)
Roll no. - 2223448

o/p—







Name: Armaan Patnaik
Ly 28(2021)
Roll no. - 2223448

Q.3

Cloud Shell Editor

cloudshell

GNU nano 7.2

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5

pthread_mutex_t fork_mutex[N];

void *philosopher(void *arg)
{
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % N;

    printf("Philosopher %d is thinking\n", id + 1);
    usleep(100000);

    printf("Philosopher %d is hungry\n", id + 1);

    pthread_mutex_lock(&fork_mutex[left]);
    printf("Philosopher %d picked up left fork\n", id + 1);

    usleep(200000);

    printf("Philosopher %d is waiting for right fork\n", id + 1);
    pthread_mutex_lock(&fork_mutex[right]);

    printf("Philosopher %d is eating\n", id + 1);
    usleep(100000);

    pthread_mutex_unlock(&fork_mutex[right]);
    pthread_mutex_unlock(&fork_mutex[left]);

    return NULL;
}

int main()
{
    pthread_t philosophers[N];
    int id[N];

    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&fork_mutex[i], NULL);
        id[i] = i;
```

```c
    printf("Philosopher %d picked up left fork\n", id + 1);

    usleep(200000);

    printf("Philosopher %d is waiting for right fork\n", id + 1);
    pthread_mutex_lock(&fork_mutex[right]);

    printf("Philosopher %d is eating\n", id + 1);
    usleep(100000);

    pthread_mutex_unlock(&fork_mutex[right]);
    pthread_mutex_unlock(&fork_mutex[left]);

    return NULL;
}

int main()
{
    pthread_t philosophers[N];
    int id[N];

    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&fork_mutex[i], NULL);
        id[i] = i;
    }

    for (int i = 0; i < N; i++)
        pthread_create(&philosophers[i], NULL, philosopher, &id[i]);

    for (int i = 0; i < N; i++)
        pthread_join(philosophers[i], NULL);

    return 0;
}
```

Name: Armaan Patnaik
Ly 28(2021)
Roll no. - 2223448

o/p—

```
armaanpatnaik2004@cloudshell:~$ nano q3_deadlock.c
armaanpatnaik2004@cloudshell:~$ gcc q3_deadlock.c -o q3_deadlock -pthread
armaanpatnaik2004@cloudshell:~$ ./q3_deadlock
Philosopher 1 is thinking
Philosopher 5 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 1 is hungry
Philosopher 1 picked up left fork
Philosopher 3 is hungry
Philosopher 3 picked up left fork
Philosopher 4 is hungry
Philosopher 4 picked up left fork
Philosopher 2 is hungry
Philosopher 5 is hungry
Philosopher 5 picked up left fork
Philosopher 2 picked up left fork
Philosopher 1 is waiting for right fork
Philosopher 4 is waiting for right fork
Philosopher 5 is waiting for right fork
Philosopher 2 is waiting for right fork
Philosopher 3 is waiting for right fork
^C
armaanpatnaik2004@cloudshell:~$ nano q3_deadlock.c
armaanpatnaik2004@cloudshell:~$
```

Name: Armaan Patnaik
Ly 28(2021)
Roll no. - 2223448

Q.4

Cloud Shell Editor

cloudshell

GNU nano 7.2

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>

#define CUSTOMERS 10

int waiting_chairs;
int waiting = 0;
int served = 0;
int rejected = 0;
int sleeping = 0;
int max_queue = 0;

pthread_mutex_t mutex;
sem_t customers;
sem_t barber_ready;

void *barber(void *arg)
{
    while (served + rejected < CUSTOMERS) {

        pthread_mutex_lock(&mutex);

        if (waiting == 0) {
            printf("Barber is sleeping\n");
            sleeping++;
        }

        pthread_mutex_unlock(&mutex);

        sem_wait(&customers);

        pthread_mutex_lock(&mutex);

        waiting--;
        served++;

        printf("Barber woke up\n");
        printf("Barber is serving a customer | Waiting: %d\n", waiting);

        pthread_mutex_unlock(&mutex);
```

Cloud Shell Editor

cloudshell

GNU nano 7.2

```
        pthread_mutex_unlock(&mutex);

        sem_post(&barber_ready);

        usleep(200000);
    }

    return NULL;
}

void *customer(void *arg)
{
    int id = *(int *)arg;

    usleep(rand() % 300000);

    pthread_mutex_lock(&mutex);

    printf("Customer %d arrived\n", id);

    if (waiting < waiting_chairs) {

        waiting++;

        if (waiting > max_queue)
            max_queue = waiting;

        printf("Customer %d is waiting | Occupied chairs: %d\n",
               id, waiting);

        pthread_mutex_unlock(&mutex);

        sem_post(&customers);
        sem_wait(&barber_ready);

        printf("Customer %d got a haircut and left\n", id);

    } else {

        rejected++;

        printf("Customer %d left because no chair was available\n", id);
```

Name: Armaan Patnaik
Ly 28(2021)
Roll no. - 2223448

Cloud Shell Editor

cloudshell

```
  GNU nano 7.2
    }

    return NULL;
}

int main(int argc, char *argv[])
{
    pthread_t barber_thread, customer_threads[CUSTOMERS];
    int id[CUSTOMERS];

    /* Default: 3 waiting chairs */
    waiting_chairs = 3;

    if (argc > 1)
        waiting_chairs = atoi(argv[1]);

    srand(time(NULL));

    pthread_mutex_init(&mutex, NULL);
    sem_init(&customers, 0, 0);
    sem_init(&barber_ready, 0, 0);

    pthread_create(&barber_thread, NULL, barber, NULL);

    for (int i = 0; i < CUSTOMERS; i++) {
        id[i] = i + 1;
        pthread_create(&customer_threads[i], NULL,
                       customer, &id[i]);
    }

    for (int i = 0; i < CUSTOMERS; i++)
        pthread_join(customer_threads[i], NULL);

    pthread_join(barber_thread, NULL);

    printf("\n----- FINAL RESULTS -----\n");
    printf("Customers served: %d\n", served);
    printf("Customers rejected: %d\n", rejected);
    printf("Times barber slept: %d\n", sleeping);
    printf("Maximum queue length: %d\n", max_queue);

    return 0;
}
```

o/p—

Cloud Shell Editor

cloudshell

```
Producer 2 produced 27 | Buffer count: 1
Producer 1 produced 55 | Buffer count: 2
armaanpatnaik2004@cloudshell:~$ nano question4.c
armaanpatnaik2004@cloudshell:~$ gcc question4.c -o question4 -pthread
armaanpatnaik2004@cloudshell:~$ ./question4 1
Barber is sleeping
Customer 9 arrived
Customer 9 is waiting | Occupied chairs: 1
Barber woke up
Barber is serving a customer | Waiting: 0
Customer 9 got a haircut and left
Customer 1 arrived
Customer 1 is waiting | Occupied chairs: 1
Customer 2 arrived
Customer 2 left because no chair was available
Customer 4 arrived
Customer 4 left because no chair was available
Customer 10 arrived
Customer 10 left because no chair was available
Customer 3 arrived
Customer 3 left because no chair was available
Customer 5 arrived
Customer 5 left because no chair was available
Customer 6 arrived
Customer 6 left because no chair was available
Customer 8 arrived
Customer 8 left because no chair was available
Barber woke up
Barber is serving a customer | Waiting: 0
Customer 1 got a haircut and left
Customer 7 arrived
Customer 7 is waiting | Occupied chairs: 1
Barber woke up
Barber is serving a customer | Waiting: 0
Customer 7 got a haircut and left

----- FINAL RESULTS -----
Customers served: 3
Customers rejected: 7
Times barber slept: 1
Maximum queue length: 1
armaanpatnaik2004@cloudshell:~$ ./question4 2
Barber is sleeping
Customer 7 arrived
Customer 7 is waiting | Occupied chairs: 1
Barber woke up
Barber is serving a customer | Waiting: 0
Customer 7 got a haircut and left
```

Cloud Shell Editor

cloudshell

```
----- FINAL RESULTS -----
Customers served: 3
Customers rejected: 7
Times barber slept: 1
Maximum queue length: 1
armaanpatnaik2004@cloudshell:~$ ./question4 2
Barber is sleeping
Customer 7 arrived
Customer 7 is waiting | Occupied chairs: 1
Barber woke up
Barber is serving a customer | Waiting: 0
Customer 7 got a haircut and left
Customer 1 arrived
Customer 1 is waiting | Occupied chairs: 1
Customer 8 arrived
Customer 8 is waiting | Occupied chairs: 2
Customer 2 arrived
Customer 2 left because no chair was available
Customer 3 arrived
Customer 3 left because no chair was available
Customer 10 arrived
Customer 10 left because no chair was available
Customer 9 arrived
Customer 9 left because no chair was available
Customer 6 arrived
Customer 6 left because no chair was available
Barber woke up
Barber is serving a customer | Waiting: 1
Customer 1 got a haircut and left
Customer 4 arrived
Customer 4 is waiting | Occupied chairs: 2
Customer 5 arrived
Customer 5 left because no chair was available
Barber woke up
Barber is serving a customer | Waiting: 1
Customer 8 got a haircut and left
Barber woke up
Barber is serving a customer | Waiting: 0
Customer 4 got a haircut and left

----- FINAL RESULTS -----
Customers served: 4
Customers rejected: 6
Times barber slept: 1
Maximum queue length: 2
armaanpatnaik2004@cloudshell:~$ ./question4 3
Barber is sleeping
```

Name: Armaan Patnaik
Ly 28(2021)
Roll no. - 2223448

